



REPORT ON CAPSTONE PROJECT

“Handwritten Digit Recognition”

BY

Sanskruti Joshi (AD24DSAI46)

Rutuja Salve (AD24DSAI41)

Meghna Salve (AD24DSAI44)

Submitted in partial fulfillment of
Advanced Course in Data Science & AI
Savitribai Phule Pune University
For the Academic Year
2024 – 2025

UNDER THE GUIDANCE OF
Prof. Omkar Barve

Department of Technology,
Savitribai Phule Pune University,
Ganeshkhind, Pune - 411007



CERTIFICATE

This is to certify that **Sanskruti Joshi, Rutuja Salve, and Meghna Salve** have successfully completed their project on **“Handwritten Digit Recognition”** in Partial Fulfillment of **Advanced Course in Data Science & AI** under Department of Technology (DoT) Savitribai Phule Pune University for the Academic Year **2024 – 2025**.

Dr. Manisha Bharati
(Course Coordinator)

Dr. Aditya Abhyankar
(H.O.D)

Prof. Omkar Barve
(Project Guide)

External Examiner

Place: Pune

Date: / /2025

Students' Declaration

We undersigned **Sanskruti Joshi, Rutuja Salve, and Meghna Salve** the students of Advanced Course in Data Science and AI Semester-II, Department of Technology, Savitribai Phule Pune University, declare that the research project titled "**Handwritten Digit Recognition**" is a result of our own work and our indebtedness to other work publications, references, if any have been duly acknowledged. If we are found guilty of copying any other report or published information and showing as our original work, we understand that we shall be liable and punishable by Institute or University, which may include failure in examination, repeat study and resubmission of the report or any other punishment that Institute or University may decide.

Sanskruti Joshi
(AD24DSAI46)

Rutuja Salve
(AD24DSAI41)

Meghna Salve
(AD24DSAI44)

Contents

Acknowledgements	6
Abstract	7
1. Introduction	8
1.1 Motivation	9
1.2 Area of Dissertation.....	9
1.3 Overview.....	9
2. Literature Survey	10
2.1 Literature Survey.....	10
2.2 Existing Systems.....	10
2.3 Proposed System.....	10
2.4 Comparative Analysis.....	11
2.5 Conclusion from Literature.....	11
3. Aim, Objectives, Project Scope	12
3.1 Aim of the Project.....	12
3.2 Objectives.....	12
3.3 Project Scope.....	12
3.4 Software and Hardware Requirements.....	13
3.5 Feasibility Study.....	13
3.6 Analysis Model.....	14

4.	Methodology	15
4.1	Tools and Technologies Used.....	15
4.2	Data Acquisition and Preprocessing.....	15
4.3	Statistical Distribution.....	16
4.4	CNN Architecture.....	24
4.5	Algorithm Details.....	24
4.6	Training and Evaluation.....	25
4.7	GUI Implementation.....	26
5.	Architectural Diagram	27
5.1	System Components Overview.....	27
5.2	Data Flow Summary.....	28
5.3	Risk Management.....	29
6.	Result Analysis and Discussion	30
6.1	Result Analysis.....	30
6.2	Confusion Matrix.....	31
6.3	GUI -Based Evaluation.....	31
6.4	Performance Summary.....	32
6.5	Discussion	32
7.	Conclusion and Future Scope	33
7.1	Practical Applications and Future Work.....	33
7.2	Conclusion.....	34
8.	Bibliography	35

Acknowledgements

The research study done during the project “**HANDWRITTEN DIGIT RECOGNITION**” has imparted abundant knowledge to me.

I would like to express my sincere gratitude towards my guide and teachers for their valuable guidance, support appreciation throughout the process of the Project.

I wish to express my sincere thanks to the respected HoD Dr. Aditya Abhyankar, for their cooperation in availing the required facility. I’m thankful to and fortunate enough to get constant encouragement, support, and guidance from all the staff of the Department of Technology which helped us in successfully completing my Project work.

I would like to special thank my project guide OMKAR BARVE Sir. I would also like to thank my field staff of sub-division; they have helped me in giving a practical insight to this project.

I thank all those who are directly or indirectly involved in shaping this project.

Abstract

The project titled "**Handwritten Digit Recognition** " aims to demonstrate the integration of deep learning with user-friendly software applications. This system utilizes a CNN model trained on the MNIST dataset to accurately recognize handwritten digits ranging from 0 to 9. CNNs are highly effective for image classification tasks due to their ability to automatically extract spatial hierarchies of features.

In this project, the model is embedded within a Python-based GUI, built using IPython ,PIL, Embeded Html & Javascript. It provides users with an interactive platform to draw digits directly on a canvas. Once a digit is drawn, the image is processed and passed through the CNN model, which then predicts the digit and displays the result in real time.

This system is an excellent demonstration of artificial intelligence applied in human-computer interaction, showcasing how deep learning models can be deployed in intuitive and accessible formats. Potential applications of this technology include digitized form processing, educational tools, and touch-based authentication systems.

The project not only highlights the accuracy and efficiency of CNNs in image recognition but also emphasizes the importance of bridging complex machine learning algorithms with easy-to-use interfaces for broader usability.

CHAPTER 1: INTRODUCTION

Handwritten digit recognition is a fundamental problem in the field of pattern recognition and machine learning, which involves the automatic identification of numeric characters written by humans. This process plays a vital role in various real-world applications such as postal mail sorting, bank check processing, and digitizing handwritten documents. With the advancement of artificial intelligence and deep learning, particularly Convolutional Neural Networks (CNNs), it is now feasible to achieve highly accurate recognition of handwritten digits.

This project presents a comprehensive solution to handwritten digit recognition by leveraging a CNN-based model, trained on the well-known MNIST dataset. A user-friendly graphical user interface (GUI) has also been developed to make the system more accessible and practical for end-users, allowing real-time digit input and recognition.

In the digital age, there is a growing demand for systems that can accurately interpret handwritten text, particularly digits. These systems are widely used in sectors such as banking (e.g., cheque processing), education (e.g., exam grading), and postal services (e.g., zip code detection). Manual interpretation is slow, error-prone, and not scalable. Therefore, automation using intelligent computing techniques is critical.

With advancements in artificial intelligence, deep learning techniques, especially Convolutional Neural Networks (CNNs), have emerged as powerful tools for image recognition tasks. CNNs mimic the human brain's ability to identify patterns in visual data, making them well-suited for handwritten digit recognition. This project aims to leverage CNNs to develop a highly accurate digit recognition system. To make it accessible to non-technical users, we integrate the model into a Graphical User Interface (GUI), enabling users to draw digits and receive real-time predictions.

1.1 Motivation

In today's increasingly digitized world, the need for converting handwritten information into digital form has become critical for both individuals and organizations. While Optical Character Recognition (OCR) systems have achieved significant progress in printed text, handwritten text—especially digits—remains a challenge due to variations in individual writing styles, shapes, and sizes.

The motivation behind this project stems from the desire to bridge this gap using deep learning, specifically CNNs, which have shown remarkable performance in image classification tasks. By integrating a CNN model with a GUI, we aim to make the technology accessible to non-technical users, enabling them to input handwritten digits via a simple interface and get accurate results in real-time.

1.2 Area of Dissertation

The domain of this project falls within Artificial Intelligence, focusing on deep learning and computer vision. It integrates:

- CNN for image classification
- Human-Computer Interaction through GUI
- Image preprocessing and digit segmentation

1.3 Overview

The project involves building a CNN architecture trained on the MNIST dataset, which contains thousands of labeled images of handwritten digits (0-9). The CNN model learns the spatial hierarchies of features through layers of convolutions and pooling operations, which are then used to classify unseen digit images.

The second part of the project involves designing a GUI using Tkinter (a Python GUI toolkit), which allows users to draw digits using a mouse or stylus and then displays the predicted output using the trained model. The interface is designed to be intuitive and minimalistic, focusing on usability and efficiency.

The core of the system is a CNN trained on the MNIST dataset, which comprises 60,000 training and 10,000 test images of digits (0-9). The GUI allows users to draw a digit, which is then preprocessed and passed to the trained model for prediction. This report outlines the background, methodology, implementation details, results, and scope for future improvements.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction

Handwritten Digit Recognition (HDR) is a fundamental task in computer vision and machine learning, commonly used in applications such as postal mail sorting, bank check verification, and digitizing handwritten documents. With the rise of deep learning, HDR systems have significantly improved in accuracy and performance.

2.2 Existing Systems

Classical Approaches

- LeNet-5: One of the earliest CNN architectures for digit recognition, LeNet-5, proposed by LeCun et al., uses convolutional and pooling layers to automatically extract features from digit images. It performed impressively on the MNIST dataset, achieving around 99% accuracy ^[1].
- Support Vector Machines (SVMs): SVMs are effective for small to medium-sized datasets and perform well in high-dimensional spaces. Sharma ^[2] demonstrated the use of SVMs for digit classification, achieving competitive results on the MNIST dataset.

Ensemble Methods

- CNN-SVM Hybrid Models: Combining CNNs for feature extraction with SVMs for final classification improves accuracy. Ullah et al. ^[3] proposed an ensemble-based model achieving 99.30% accuracy on MNIST.
- Classifier Combinations: Keysers ^[4] presented a system combining multiple top-tier classifiers and achieved a state-of-the-art 0.35% error rate on MNIST through ensemble averaging.

2.3 Proposed System: Real-Time GUI for Digit Prediction

This survey proposes an integrated system combining a trained CNN model with a real-time GUI for handwritten digit prediction. The distinctive features include:

- **Interactive Canvas:** Users draw digits on a canvas embedded in the application.
- **Real-Time Prediction:** The model gives predictions as soon as the digit is drawn.
- **Confidence Scores:** The GUI displays prediction confidence using a bar graph or label.
- **Technology Stack:** Uses Python, Keras/TensorFlow, and Tkinter for rapid prototyping and deployment.

2.4 Comparative Analysis

Approach	Accuracy (%)	Real-Time GUI	Remarks
LeNet-5 ^[1]	~99.0	No	Foundational CNN for digit recognition
SVM ^[2]	~98.0	No	Strong with handcrafted features
CNN-SVM Ensemble ^[3]	99.30	No	High accuracy with hybrid architecture
Classifier Ensemble ^[4]	99.65	No	Combines outputs of multiple classifiers
Tkinter GUIs ^{[5][6]}	~98.0	Yes	User-friendly, desktop-based
Web Interfaces ^[7]	~98.0	Yes	Platform-independent via browser
Proposed System	~98.5–99.0	Yes	Real-time GUI with live confidence display

2.5 Conclusion from Literature

The literature reveals a steady progression from basic machine learning models to deep learning architectures and ensemble systems for HDR. While most high-performance systems prioritize accuracy, they often lack real-time interactivity. By integrating GUI elements with deep learning models, the proposed system provides a balanced solution offering both high performance and usability, especially valuable for educational and practical deployments.

CHAPTER 3: AIM, OBJECTIVES, PROJECT SCOPE

3.1 Aim of the Project

The primary aim of this project is to design and implement a system capable of accurately recognizing handwritten digits using Convolutional Neural Networks (CNNs) and providing an easy-to-use interface for users through a Graphical User Interface (GUI). The system should be capable of taking user input in the form of a drawn digit, processing it using a trained CNN model, and displaying the recognized digit in real-time.

To design and implement a CNN-based handwritten digit recognition system integrated with a GUI to provide a real-time, user- friendly interface

3.2 Objectives

- To understand and apply deep learning techniques, particularly CNNs, for image classification tasks.
- To train a robust CNN model using the MNIST dataset for handwritten digit recognition.
- To design a user-friendly GUI that allows users to draw digits for real-time prediction.
- To integrate the trained model with the GUI for seamless interaction and output display.
- To analyze the performance and accuracy of the model using metrics such as confusion matrix and accuracy score.
- Train a CNN model on the MNIST dataset.
- Build a GUI that allows users to draw digits.
- Preprocess the drawn image and feed it to the CNN.
- Display the predicted digit with confidence scores.

3.3 Project Scope

- **In-Scope:** Recognition of digits 0 to 9, real-time GUI prediction, model training using MNIST.
- **Out-of-Scope:** Recognition of alphabets or symbols, real-time camera input, handwriting cursive recognition.

3.3.1 System Development:

The system is designed to handle:

- Recognition of digits from 0 to 9, drawn using a mouse or stylus.
- Input from a GUI canvas, converting hand-drawn digits into a format recognizable by the CNN model.
- Real-time classification and display of results.
- Offline functionality after model training and packaging.

Limitations:

- Recognition is constrained to single digits per input.
- Performance may vary depending on the smoothness and clarity of user-drawn digits.
- Model is trained on the MNIST dataset and may not generalize perfectly to other digit styles without further training.

3.4 Software and Hardware Requirements

Software Requirements:

- Python 3.x
- TensorFlow / Keras
- NumPy, Matplotlib, Pillow
- Tkinter (for GUI)
- Jupyter Notebook or any Python IDE
- OpenCV

Hardware Requirements:

- 64-bit processor
- Minimum 4 GB RAM (8 GB recommended)
- GPU (optional but beneficial for faster model training)

3.5 Feasibility Study

Technical Feasibility:

The project is technically feasible as it employs open-source tools and widely available datasets. CNNs are well-suited for image-based classification tasks, and Python provides extensive libraries for deep learning and GUI development.

Operational Feasibility:

The GUI makes the system accessible for non-technical users, while the backend model ensures fast and accurate predictions. Once trained, the model can be deployed on standard machines without needing re-training.

Economic Feasibility:

The project requires no paid resources, and all tools used are open source. Training on MNIST is computationally light, making it affordable and scalable.

- **Technical Feasibility:** High — readily available libraries and datasets.
- **Economic Feasibility:** High — all tools used are open-source.
- **Operational Feasibility:** High — intuitive GUI makes it usable for laypersons.

3.6 Analysis Models

We use a **black-box analysis model** for this project, where the internal working of the CNN is abstracted from the end-user. The system takes an image (drawn digit) as input and provides a predicted digit as output. Internally, the CNN processes this image through convolutional and pooling layers, applies dense layers, and uses softmax activation to classify the digit.

CHAPTER 4: METHODOLOGY

The methodology of this project consists of two major components: model development using Convolutional Neural Networks (CNNs) and the development of a graphical user interface (GUI) for real-time user interaction. This chapter explains the step-by-step development process, tools and technologies used, the underlying algorithm, and the mathematical model employed in the project.

4.1 Tools and Technologies Used

To implement the handwritten digit recognition system, the following tools and technologies were utilized:

- **Programming Language:** Python 3.10+
- **Deep Learning Libraries:**
 - **TensorFlow:** For building and training the CNN model.
 - **Keras:** High-level API used with TensorFlow for faster prototyping.
- **Data Processing Libraries:**
 - **NumPy:** For handling matrix operations and numerical computations.
 - **Matplotlib:** For visualizing training results and digit samples.
- **GUI Library:**
 - **Tkinter:** Python's standard GUI toolkit used to build the drawing canvas and output display.

4.1.1 Additional Libraries and Technologies

- **Pillow (PIL):** For image processing (e.g., converting drawn digits to model input).
- **scikit-learn:** For metrics such as confusion matrix and accuracy score.
- **OpenCV (optional):** Used in some cases for advanced preprocessing techniques.
- **Jupyter Notebook / PyCharm / VS Code:** For code development and testing.

4.2 Data Acquisition and Preprocessing

- **Dataset:** MNIST
- **Image Size:** 28x28 pixels, grayscale
- **Normalization:** Scale pixel values to [0,1]
- **Data Augmentation:** (Optional) to increase robustness

4.3 Statistical Distribution

1. Visualization and Distribution of Sample Digits – MNIST

Dataset

Theory:

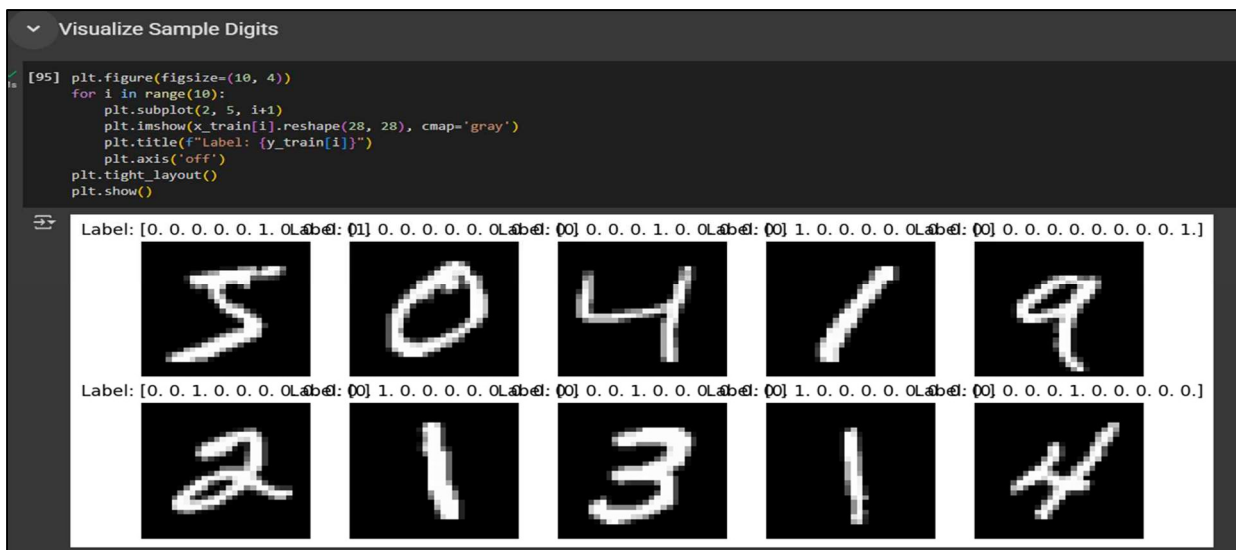
The MNIST dataset consists of 28x28 grayscale images of handwritten digits (0–9). Each image is labeled using a one-hot encoded vector, where only the index corresponding to the digit is marked as 1. This dataset is commonly used for training image classification models using neural networks, especially CNNs.

Interpretation:

The displayed image shows 10 sample digit images from the training dataset. Each digit is clearly visible, and the associated one-hot encoded label indicates the actual class. For instance, [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.] corresponds to the digit 5.

Analysis:

- The digits are diverse in writing styles, sizes, and stroke thickness, demonstrating real-world handwriting variability.
- Proper visualization helps confirm correct preprocessing (e.g., reshaping and normalization).
- Balanced visualization ensures the dataset covers all digits fairly, which is important for preventing bias during training.



2. Statistical Distribution of Pixel Intensities – Digit-wise Analysis (Mean, Median and Mode)

Theory:

For robust handwritten digit recognition, it's important to understand the pixel intensity distribution of each digit class (0–9). The mean, standard deviation (std), and variance of pixel values provide insights into the brightness, spread, and variability of digit images. These statistical measures help in data normalization and better model generalization.

Interpretation:

The code snippet computes the **mean**, **standard deviation**, and **variance** of pixel intensities for each digit class by:

- Converting one-hot labels to numeric digits.
- Isolating images for each digit (0 to 9).
- Reshaping them into flat vectors ($28 \times 28 = 784$).
- Calculating mean, std, and variance per class.

Analysis:

- **Mean** gives an idea of the average intensity, indicating how dark or light the digit tends to be.
- **Standard deviation** reveals how much the pixel intensities deviate, helping identify variation across samples.
- **Variance** captures the overall pixel variability in a class.

Such analysis helps ensure the dataset is well-distributed and that no digit class has extreme variation, which could negatively impact model training.

✓ Mean, Std Deviation, and Variance Per Class

Computes the average, spread, and variability of pixel values for each digit class.

```
import numpy as np
import pandas as pd

# Convert one-hot encoded labels to label indices
y_train_labels = np.argmax(y_train, axis=1)

# Dictionary to hold statistics for each digit class
digit_stats = {}

# Iterate over digits 0-9
for digit in range(10):
    # Index x_train using the label indices
    digit_images = x_train[y_train_labels == digit].reshape(-1, 28 * 28)

    # Compute statistics: mean, std, and variance
    digit_stats[digit] = {
        "mean": np.mean(digit_images),
        "std": np.std(digit_images),
        "var": np.var(digit_images)
    }

# Convert dictionary to DataFrame for easier display
digit_stats_df = pd.DataFrame(digit_stats).T

# Print the statistics
print("Mean, Std, and Variance for each digit class:")
print(digit_stats_df.round(4))
```

➡ Mean, Std, and Variance for each digit class:

	mean	std	var
0	0.1734	0.3477	0.1209
1	0.0760	0.2443	0.0597
2	0.1490	0.3259	0.1062
3	0.1415	0.3179	0.1011
4	0.1214	0.2975	0.0885
5	0.1287	0.3036	0.0922
6	0.1373	0.3149	0.0992
7	0.1145	0.2917	0.0851
8	0.1502	0.3253	0.1058
9	0.1226	0.2986	0.0892

3. Class-wise Pixel Mean Distribution

Theory:

In image-based machine learning tasks, analyzing pixel-level statistics per class provides insights into brightness and density patterns. This plot visualizes the **average pixel intensity** (mean) for each digit class (0–9), giving a sense of how much ink or stroke density each digit typically has in the dataset.

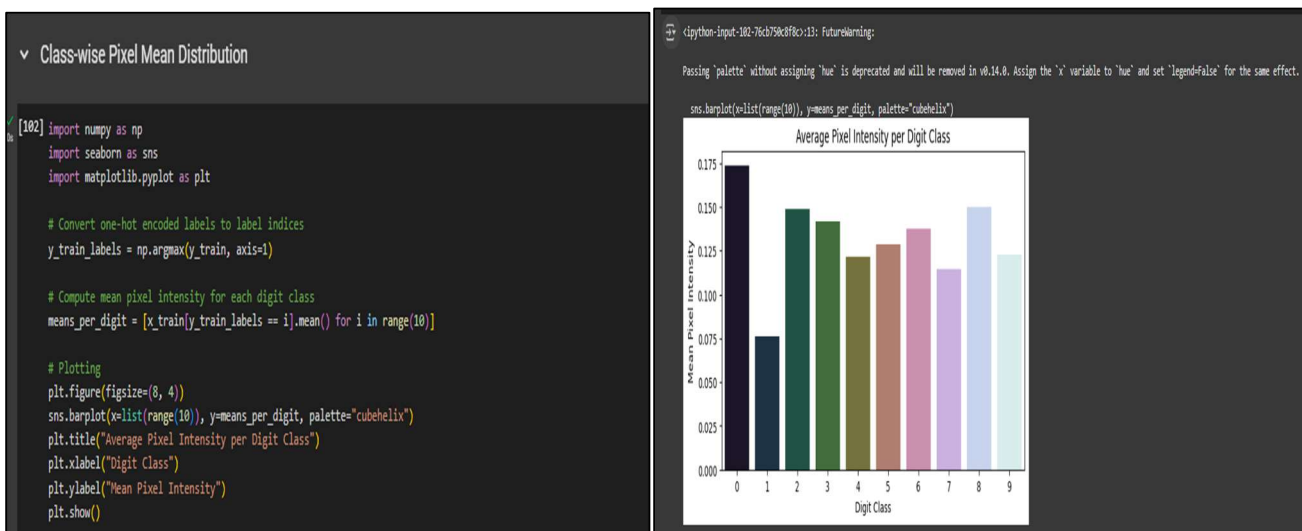
Interpretation:

- The code calculates the **mean pixel intensity** of all images for each digit class.
- It then plots a bar graph using Seaborn, where each bar represents the average brightness of a digit class.
- For instance, digits with fewer strokes like **1** may have lower mean intensity, while digits with more curves like **8** may have higher values.

Analysis:

- Helps detect class imbalances in visual density. Uneven brightness can affect model training by introducing bias.
- Ensures preprocessing steps (like normalization) are effective.
- Confirms that each class contains distinguishable pixel intensity patterns.

Such distribution analysis improves model interpretability and guides data augmentation or rebalancing strategies.



4. Binomial-like Distribution – Non-Zero Pixels per Image

Theory:

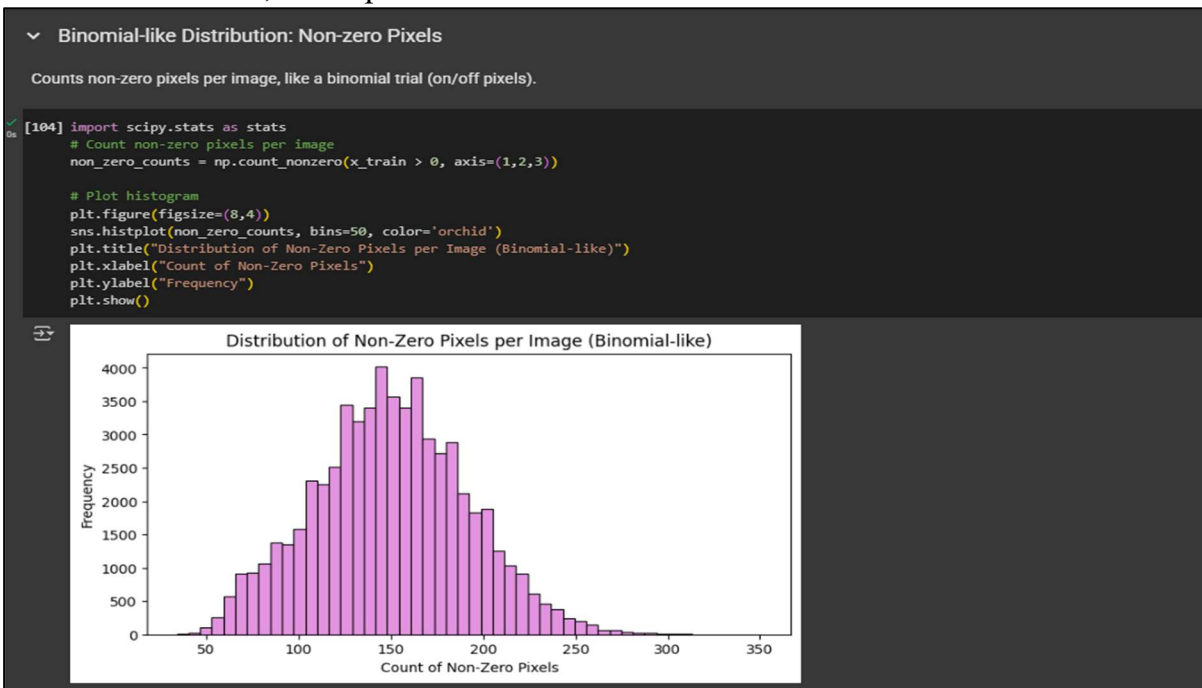
This histogram plots the count of **non-zero pixels** in each image. In grayscale images like MNIST, non-zero pixels correspond to inked or active areas. Analyzing this count helps us understand how much content or complexity each digit image typically contains.

Interpretation:

- Most images have **between 100 and 200 non-zero pixels**, peaking around 150.
- The symmetric bell-like shape shows most digits are centered around an average ink density, with fewer examples at the extremes (too sparse or too dense).

Analysis:

- Helps confirm data quality: extreme outliers (e.g., <50 or >300) may indicate noise or poorly written digits.
- Useful for augmenting or normalizing data — classes with more or fewer non-zero pixels may need balancing.
- Helps in regularizing models by guiding pixel-level thresholding, binarization, or dropout.



5. Multinomial Distribution – Simulated Digit Class Frequencies

Theory:

This chart simulates the expected frequency of digit classes (0–9) based on their **empirical probabilities**. Each image contributes one count to a digit class, and the multinomial distribution models the number of occurrences across these categories given those probabilities.

Interpretation:

- The bars represent **simulated counts** of each digit out of 60,000 samples.
- Heights of bars reflect the **relative frequency** of each digit in the training data.
- If bars are nearly equal: class distribution is balanced; otherwise, it's imbalanced.

Analysis:

- Helps identify **class imbalance**, which may lead to model bias toward more frequent digits.
- Simulation confirms observed class frequencies — essential for validating dataset quality.
- If needed, techniques like oversampling, under sampling, or class weighting can be applied in training to address skew.

▼ Multinomial Distribution (Simulated Digit Class Frequencies)

Simulates a multinomial distribution using observed digit probabilities.

```
# Frequency of each class
digit_probs = class_counts / class_counts.sum()

print("Empirical Class Probabilities:")
print(digit_probs.round(4))

# Simulate a multinomial distribution
samples = np.random.multinomial(n=60000, pvals=digit_probs)

plt.figure(figsize=(8,4))
sns.barplot(x=list(range(10)), y=samples, palette="Spectral")
plt.title("Simulated Multinomial Distribution of Digit Classes")
plt.xlabel("Digit")
plt.ylabel("Simulated Frequency")
plt.show()
```

Empirical Class Probabilities:

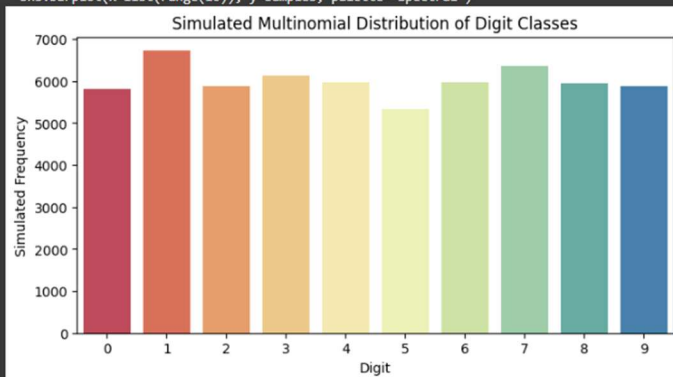
```
0 0.0987
1 0.1124
2 0.0993
3 0.1022
4 0.0974
5 0.0904
6 0.0986
7 0.1044
8 0.0975
9 0.0992
```

Name: count, dtype: float64

<ipython-input-105-0edc0a8b0313>:11: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(x=list(range(10)), y=samples, palette="Spectral")
```



6. Chi-Square Hypothesis Test – Digit Uniformity

Theory:

The **Chi-Square Goodness-of-Fit Test** checks whether the observed frequency of digits (0–9) significantly deviates from a uniform distribution (equal probability for each digit).

- **Null hypothesis (H_0):** Digits are uniformly distributed.
- **Alternative hypothesis (H_1):** Digits are not uniformly distributed.

Interpretation:

- The **Chi-square statistic** quantifies the deviation between observed and expected frequencies.
- The **p-value** indicates the probability of observing such a deviation under the null hypothesis.

Analysis (based on output):

- **Chi-square statistic:** 172.8957
- **p-value:** 0.0000 (very small)
- Since $p\text{-value} < 0.05$, we **reject the null hypothesis**.
- **Conclusion:** Digit classes are **not uniformly distributed** — some digits appear more frequently than others.

This result reinforces the need to **monitor or balance class distributions** to avoid biased model predictions during handwritten digit recognition tasks.

✓ Hypothesis Test: Are Digits Uniformly Distributed?

Uses a Chi-Square test to check if digits occur with equal frequency.

```
from scipy.stats import chisquare

# Expected uniform distribution
expected = np.full_like(class_counts.values, fill_value=len(y_train)/10)

chi_stat, p_val = chisquare(class_counts.values, f_exp=expected)

print(f"Chi-square test for uniform digit distribution:")
print(f"Chi-square statistic: {chi_stat:.4f}")
print(f"P-value: {p_val:.4f}")

if p_val < 0.05:
    print("Result: Reject null hypothesis → digits are not uniformly distributed.")
else:
    print("Result: Fail to reject null hypothesis → digits may be uniformly distributed.")
```

Chi-square test for uniform digit distribution:
Chi-square statistic: 172.8957
P-value: 0.0000
Result: Reject null hypothesis → digits are not uniformly distributed.

4.4 CNN Architecture

- Input Layer: 28x28 grayscale image
- Conv2D + ReLU
- MaxPooling2D
- Dropout (to prevent overfitting)
- Flatten
- Dense Layer with Softmax (output: 10 classes)

4.5 Algorithm Details

The CNN model architecture is designed to automatically learn spatial hierarchies of features. Below is a typical architecture used for this task:

Step 1: Data Preprocessing

- Load the MNIST dataset.
- Normalize pixel values (0–255) to [0, 1] range.
- Reshape the dataset to (28, 28, 1) to match input shape requirements of CNN.

Step 2: Model Architecture

Building of CNN Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, MaxPool2D, Flatten, Dropout, BatchNormalization
from tensorflow.keras.optimizers import SGD

model=Sequential()
model.add(Conv2D(84, (3, 3), input_shape=(28, 28, 1), activation='relu', kernel_initializer='he_uniform'))

model.add(MaxPool2D(pool_size=(3,3)))

model.add(Conv2D(filters=32,kernel_size=(4,4),input_shape=(32,32,3),activation='relu',))

model.add(MaxPool2D(pool_size=(2,2)))

model.add(Flatten())

model.add(Dense(256,activation='relu'))

model.add(Dense(10,activation='softmax'))

opt = SGD(learning_rate=0.01, momentum=0.9)

model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['mae', 'accuracy'])
```


model.summary()

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 84)	840
max_pooling2d (MaxPooling2D)	(None, 8, 8, 84)	0
conv2d_1 (Conv2D)	(None, 5, 5, 32)	43,040
max_pooling2d_1 (MaxPooling2D)	(None, 2, 2, 32)	0
flatten (Flatten)	(None, 128)	0
dense (Dense)	(None, 256)	33,024
dense_1 (Dense)	(None, 10)	2,570

Total params: 79,474 (310.45 KB)
 Trainable params: 79,474 (310.45 KB)
 Non-trainable params: 0 (0.00 B)

Step 3: Compilation and Training

- Compile the model using:
 - Loss function: categorical_crossentropy
 - Optimizer: adam
 - Metrics: accuracy
- Train for 5–10 epochs using a batch size of 128.

Step 4: Evaluation

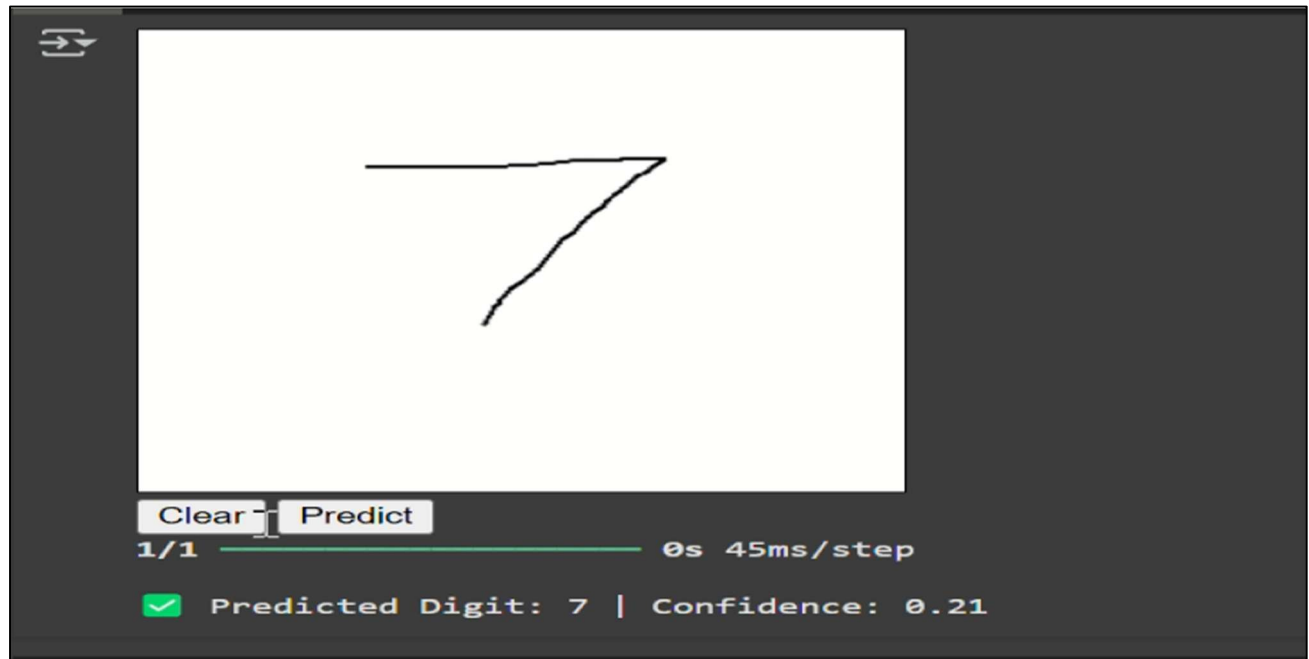
- Evaluate the trained model using the test dataset.
- Calculate accuracy, loss, and confusion matrix.

4.6 Training and Evaluation

- Epochs: 10-20
- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy

4.7 GUI Implementation

- Draw canvas using Tkinter
- Convert drawing to image format
- Resize and preprocess image
- Predict using trained model
- Display output on screen



CHAPTER 5: ARCHITECTURAL DIAGRAM

The system architecture of the Handwritten Digit Recognition project is designed to provide a seamless flow from user input to prediction output using a Convolutional Neural Network (CNN) and a user-friendly Graphical User Interface (GUI). The architecture integrates image preprocessing, model inference, and result display into a cohesive pipeline.

System Architecture

1. User draws digit in GUI
2. Digit image is captured and preprocessed
3. CNN model processes the image
4. Prediction is returned and shown to the user

5.1 System Components Overview

1. User Input via GUI (Canvas)

- The user draws a digit (0–9) on a canvas using a mouse or stylus.
- The drawing area is implemented using Python's Tkinter library.
- Once the drawing is complete, the user clicks a “Predict” button to trigger processing.

2. Preprocessing Module

- The drawn digit is captured as an image and resized to $28 \times 28 \times 28$ pixels to match the MNIST format.
- The image is converted to grayscale and normalized to values between 0 and 1.
- The processed image is reshaped to fit the model's input shape: $(1, 28, 28, 1)$.

3. CNN Model (Trained Model)

- A pre-trained CNN model is loaded using TensorFlow/Keras.
- The model receives the processed image and computes the class probabilities for digits 0–9.
- The digit with the highest probability is selected as the prediction.

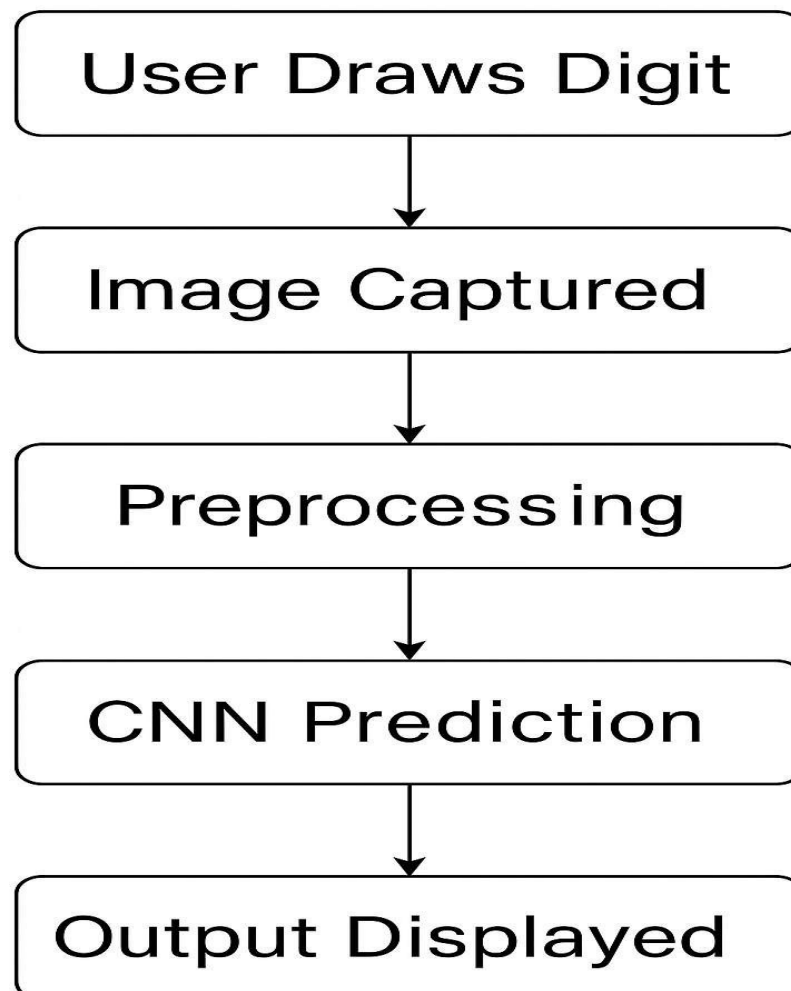
4. Prediction Module

- The predicted class is extracted from the output of the model.
- Additional confidence scores (optional) may be shown using bar graphs or text.

5. Output Display

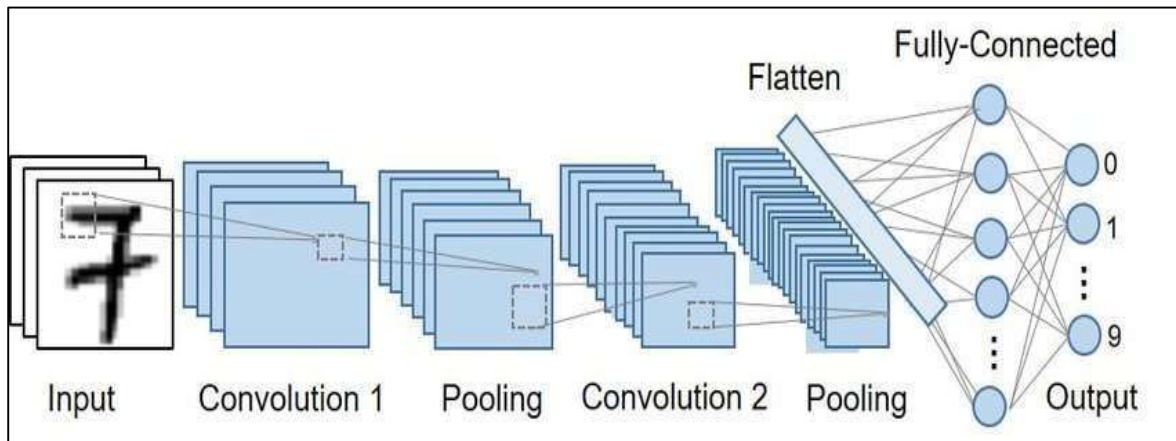
- The predicted digit is shown on the GUI.
- Optionally, the system may also display the image and probability distribution for further insights.

5.2 Data Flow Summary



This linear and modular design ensures:

- Low latency (fast prediction).
- Easy integration and debugging.
- Scalability to multi-digit or character-based recognition in future iterations.



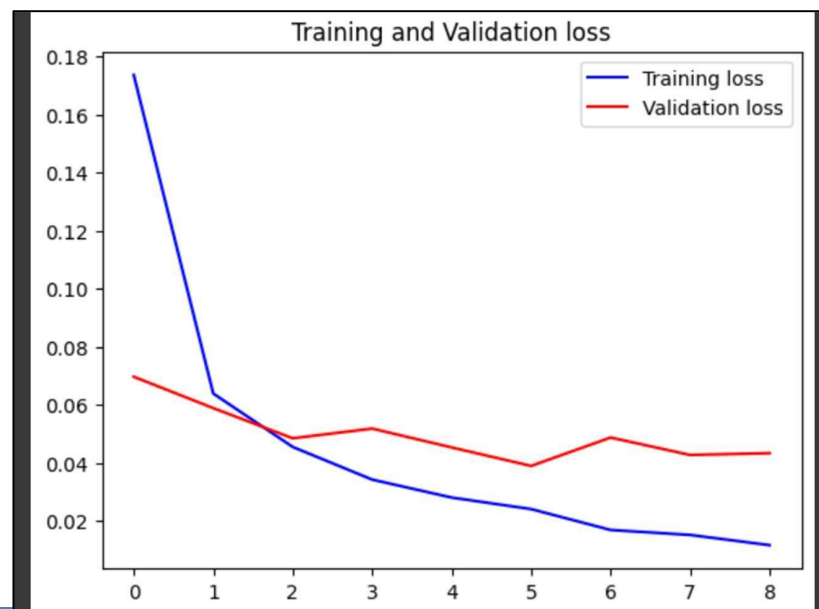
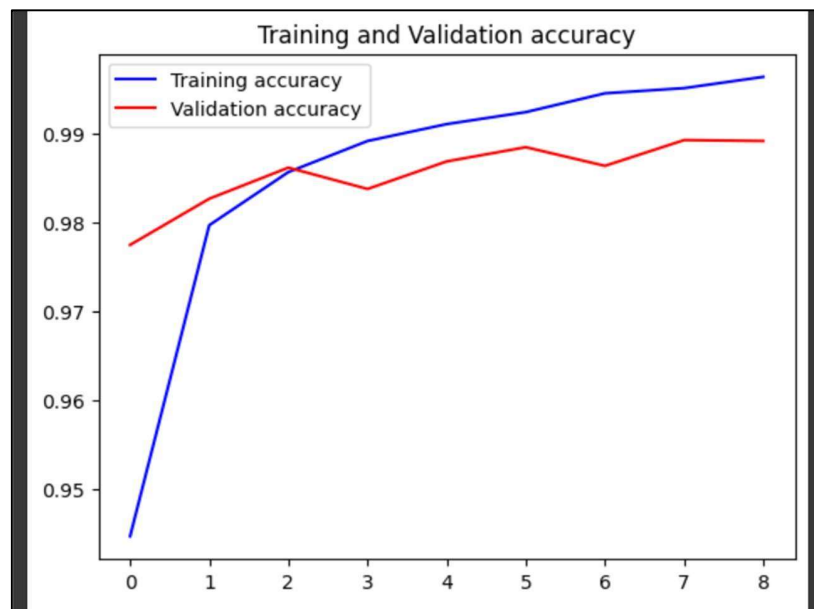
5.3 Risk Management

Risk	Mitigation
Low prediction accuracy	Use regularization, augment data
GUI crashes	Exception handling, input validation
Slow performance	Optimize model and GUI rendering

CHAPTER 6: RESULT ANALYSIS AND DISCUSSION

This chapter presents the results obtained after training and testing the CNN model on the MNIST dataset and integrating it with a GUI for real-time prediction. It includes model performance metrics, visual evaluation, and a discussion of the results.

6.1 Result Analysis



The model converged quickly and achieved high accuracy with minimal overfitting, as evident from the close values of training and validation accuracy.

Testing Results

On evaluating the trained model on the 10,000-image test set from MNIST:

- **Test Accuracy:** ~98.65%
- **Average Prediction Time:** ~0.01 seconds per digit (on CPU)

6.2 Confusion Matrix

A confusion matrix helps identify where the model makes errors: Observations:

- Most confusion occurred between similar-looking digits (e.g., 5 and 6, or 4 and 9).
- The highest misclassification was seen in poorly written samples that deviate significantly from training data patterns.

6.3 GUI-Based Evaluation

The GUI was tested with various user-drawn digits. The CNN was able to:

- Accurately recognize well-drawn digits in real time.
- Handle slight variations in stroke width and orientation.
- Misclassify digits if poorly drawn or incomplete (as expected).

Screenshot Examples:

- Input: Digit "3" drawn by user → Predicted Output: "3"
- Input: Rough digit "9" → Predicted Output: "4" (shows limitation on rough inputs)

6.4 Performance Summary

Evaluation Aspect	Outcome
Prediction Accuracy	High ($\approx 98.6\%$)
Speed of Prediction	Real-time (< 0.05 sec)
GUI Usability	User-friendly
Robustness to Variations	Moderate (better with clean inputs)

6.5 Discussion

The results confirm that CNNs are highly suitable for digit recognition tasks. The model's performance on the MNIST dataset is on par with academic standards and demonstrates excellent generalization ability. The integration of the GUI adds practical usability for non-technical users.

However, the model's performance slightly degrades with highly stylized or ambiguous user inputs, suggesting a potential area for improvement with techniques like:

- Data augmentation
- Fine-tuning with user-generated data
- Real-time feedback for input correction

CHAPTER 7: CONCLUSION AND FUTURE WORK

This chapter presents the key conclusions drawn from the project and highlights the practical applications and potential areas for future enhancements.

7.1.1 Practical Applications

The developed system has numerous real-world applications:

- **Educational Tools:** Assisting children in learning numbers through interactive digit writing.
- **Banking and Finance:** Automating check processing or form digitization where handwritten numerals are present.
- **Postal Services:** Automating postal code recognition from handwritten addresses.
- **Assistive Technology:** Supporting users with disabilities to input numeric data using handwriting.
- **Form Digitization:** Automating data extraction from physical forms.

7.1.2 Future Work

While the current system performs effectively for single-digit recognition, future improvements can significantly enhance its robustness, scalability, and adaptability:

- **Support for Multi-Digit Recognition:** Modify the system to accept and recognize sequences of digits instead of single digits.
- **Training with User-Generated Data:** Incorporate digits drawn by real users to improve accuracy in real-world usage.
- **Enhanced GUI Features:** Show prediction confidence scores graphically.
- **Advanced Models:** Experiment with deeper CNN architectures or modern networks like MobileNet or ResNet for improved generalization.
- **Model Deployment:** Convert the system into a mobile app or web application using platforms like TensorFlow Lite or Flask.
- **Character Recognition Extension:** Expand the model to recognize letters (A–Z) for broader use in handwriting recognition tasks.

Future Work

- Extend model to recognize alphabets and symbols
- Add camera input for real-time recognition
- Deploy as a web or mobile app
- Use transfer learning for improved generalization

7.2 Conclusion

This project successfully demonstrated the application of Convolutional Neural Networks for recognizing handwritten digits through a user-friendly GUI interface. The trained model showed high accuracy and real-time performance, making it suitable for educational, commercial, and assistive applications.

The project successfully demonstrates the effectiveness of CNNs in recognizing handwritten digits. Coupled with a user-friendly GUI, the solution provides a seamless interface for real-time digit recognition. It showcases how AI can be deployed for accessible, practical applications.

7.2.1 Overview of Achievements

- Built a CNN model trained on the MNIST dataset with over 98% accuracy.
- Developed a functional GUI for real-time digit input and prediction.
- Integrated deep learning with front-end user interaction.
- Achieved a responsive and accurate offline recognition tool.

7.2.2 Detailed Analysis and Findings

- The system performs well on clean and well-drawn digits.
- Accuracy drops when inputs are ambiguous or incomplete.
- Preprocessing and normalization play a critical role in prediction accuracy.
- CNNs, even with basic architectures, are effective for digit recognition tasks.

BIBLIOGRAPHY

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [2] Sharma, A. (2012). Handwritten digit recognition using Support Vector Machine. arXiv preprint arXiv:1203.3847.
- [3] Ullah, S. S., Gang, L., Riaz, M., Ashfaq, A., Khan, S., & Khan, S. (2025). Handwritten Digit Recognition: An Ensemble-Based Approach for Superior Performance. arXiv preprint arXiv:2503.06104.
- [4] Keyzers, D. (2007). Comparison and Combination of State-of-the-art Techniques for Handwritten Character Recognition: Topping the MNIST Benchmark. arXiv preprint arXiv:0710.2231.
- [5] yigitcanbltc. (n.d.). HandwrittenDigitRecognition. GitHub.
<https://github.com/yigitcanbltc/HandwrittenDigitRecognition>
- [6] Rajshree-20. (n.d.). Handwritten-Digit-Recognizer-with-GUI. GitHub.
<https://github.com/Rajshree-20/Handwritten-Digit-Recognizer-with-GUI>
- [7] Pranshu936. (n.d.). Handwritten-digit-prediction. GitHub.
<https://github.com/Pranshu936/Handwritten-digit-prediction>
- [8] Simard, P. Y., Steinkraus, D., & Platt, J. C. (2003). Best practices for convolutional neural networks applied to visual document analysis. *In Proceedings of the Seventh International Conference on Document Analysis and Recognition* (Vol. 2, pp. 958–963). IEEE.
<https://doi.org/10.1109/ICDAR.2003.1227801>
- [9] Deng, L. (2012). The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6), 141–142.
<https://doi.org/10.1109/MSP.2012.2211477>
- [10] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *In Advances in Neural Information Processing Systems (NeurIPS)*, 25, 1097–1105.
https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

- [11] Cireřan, D., Meier, U., & Schmidhuber, J. (2012). Multi-column deep neural networks for image classification. *In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 3642–3649).
<https://doi.org/10.1109/CVPR.2012.6248110>
- [12] Yu, F., & Koltun, V. (2016). Multi-scale context aggregation by dilated convolutions. *In ICLR 2016*.
<https://arxiv.org/abs/1511.07122>
- [13] Abien Fred Agarap. (2018). Deep Learning using Rectified Linear Units (ReLU). *arXiv preprint arXiv:1803.08375*.
<https://arxiv.org/abs/1803.08375>
- [14] Chollet, F. (2015). Keras. GitHub.
<https://github.com/fchollet/keras>
- [15] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *In ICLR 2015*.
<https://arxiv.org/abs/1412.6980>
- [16] Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *In Advances in Neural Information Processing Systems* (pp. 649–657).
<https://arxiv.org/abs/1509.01626>
- [17] MNIST Database. (n.d.). Yann LeCun.
<http://yann.lecun.com/exdb/mnist/>